Taller de Programación Concurrente Declarativa

Modelos y Paradigmas de Programación Escuela de Ingeniería de Sistemas y Computación — Universidad del Valle Estudiante: Raul Alzate

Ejercicio 1 — Orden de creación y evaluación de hilos

Programa analizado

```
None
declare A B C D in
thread D=C+1 end
thread C=B+1 end
thread A=1 end
thread B=A+1 end
{Browse D}
```

¿En qué orden son creados los hilos?

Los hilos son creados en el orden en que aparecen en el código fuente, ya que la instrucción `thread ... end` es ejecutada secuencialmente por el hilo principal. El orden de creación es:

1. Hilo **T1**: `D = C + 1`
2. Hilo **T2**: `C = B + 1`
3. Hilo **T3**: `A = 1`
4. Hilo **T4**: `B = A + 1`

¿En qué orden son evaluados (ejecutados) los hilos?

Una vez creados, los hilos son planificados por el motor de ejecución de Mozart. La evaluación depende de las variables de flujo de datos: un hilo se **suspende** si necesita el valor de una variable aún no ligada, y se **reanuda** automáticamente cuando esa variable es ligada por otro hilo.

El orden efectivo de evaluación es:

Paso	Hilo	Acción	Razón
1	T1	**Suspende** (D = C+1)	C no está ligada
2	T2	**Suspende** (C = B+1)	B no está ligada

Paso	Hilo	Acción	Razón
3	T3	**Ejecuta** → A = 1	No depende de ninguna variable sin ligar
4	T4	**Ejecuta** → B = 2	A ya está ligada (A=1)
5	T2	**Reanuda** → C = 3	B fue ligada (B=2)
6	T1	**Reanuda** → D = 4	C fue ligada (C=3)

`{Browse D}` muestra **4**.

Este comportamiento ilustra el **modelo de variables de flujo de datos** (*dataflow variables*): la ejecución avanza tan pronto como los datos necesarios están disponibles, logrando sincronización implícita sin necesidad de primitivas de bloqueo explícitas.

Ejercicio 2 — Comportamiento de flujo de datos con Filter

La función `Filter` definida es:

```
None
fun {Filter En F}
  case En
  of X|En2 then
    if {F X} then X|{Filter En2 F}
    else {Filter En2 F} end
  else
    nil
  end
end
```

Tanto `case` como `if` **suspenden** el hilo en que se ejecutan si no pueden decidir qué alternativa tomar (variable sin ligar).

---

a) `declare A` → `{Show {Filter [5 1 A 4 0] fun {$ X} X>2 end}}`

¿Qué pasa?

Filter comienza a recorrer la lista:

- `5 > 2` → true → 5 incluido en resultado.
- `1 > 2` → false → 1 descartado.
- Llega a `A` (sin ligar): el `if {F A}` equivale a `if A > 2`, que **no puede evaluarse** porque A es una variable no ligada. El hilo de Filter **se suspende indefinidamente**.

Dado que Filter corre en el mismo hilo que Show (llamada secuencial), **Show nunca se invoca**. El programa queda bloqueado esperando que A sea ligada.

---

b) Con hilo separado y Show inmediato

```
None
declare Sal A
thread Sal = {Filter [5 1 A 4 0] fun {$ X} X>2 end} end
{Show Sal}
```

¿Qué pasa?

Hay dos hilos:

- **Hilo Filter**: comienza a ejecutarse. Procesa 5 (incluido), 1 (descartado), llega a A y **se suspende**.
- **Hilo principal**: ejecuta `{Show Sal}` de inmediato.

`Show` despliega el valor de `Sal` **tal como está en el momento de invocación**. Como Sal aún no ha sido completamente ligada (Filter está suspendido), `Show` puede mostrar:

- Una **variable sin ligar** (`_`) — si el hilo principal llega a Show antes de que Filter haya avanzado nada.
- Una **lista parcialmente ligada** — dependiendo de cuánto haya avanzado el scheduler antes de que Show sea invocado.

El resultado es **no determinístico en apariencia**: depende del orden de planificación de los hilos en ese momento de ejecución.

¿Es Filter determinístico? No, en este contexto. Aunque la función Filter en sí es determinística (produce siempre el mismo resultado para los mismos datos de entrada completamente ligados), el resultado observable a través de `Show` varía según la planificación de hilos. La no determinabilidad no está en Filter sino en **la interacción entre la concurrencia y el momento en que se invoca Show** sobre una variable parcialmente ligada.

---

c) Con `{Delay 1000}` antes de Show

```
None
declare Sal A
thread Sal = {Filter [5 1 A 4 0] fun {$ X} X>2 end} end
{Delay 1000}
{Show Sal}
```

¿Qué pasa?

Durante los 1000 ms de Delay, el hilo principal cede el control. El hilo de Filter avanza:

- Procesa 5 (incluido), 1 (descartado), llega a A → **se suspende**.

Después del Delay, `{Show Sal}` se ejecuta. En este punto Sal sigue siendo una variable **parcialmente construida** (Filter está bloqueado en A). Show mostrará algo como una variable sin ligar o una lista incompleta, dependiendo de la representación interna.

La diferencia respecto al caso b) es que aquí hay mayor probabilidad de que Filter haya avanzado hasta el punto de bloqueo antes de que Show sea llamado, haciendo el comportamiento más **predecible** en la práctica, pero conceptualmente sigue siendo indeterminado.

---

d) Con `thread A=6 end` y `{Delay 1000}`

```
None
declare Sal A
thread Sal = {Filter [5 1 A 4 0] fun {$ X} X>2 end} end
thread A=6 end
{Delay 1000}
{Show Sal}
```

¿Qué pasa y por qué?

Se añade un tercer hilo que liga `A = 6`. La secuencia de eventos es:

1. Hilo Filter: procesa 5 (incluido), 1 (descartado), llega a A → suspende.
2. Hilo `A=6`: se ejecuta → **A queda ligada a 6**.
3. Hilo Filter: se **reanuda**. Evalúa `6 > 2` → true → 6 incluido. Continúa: 4 > 2 → true → incluido. 0 > 2 → false → descartado. Sal = `[5 6 4]`.

4. Después del Delay, `{Show Sal}` despliega **`[5 6 4]`**.

El Delay garantiza que, con alta probabilidad, todos los hilos anteriores habrán terminado antes de que Show sea invocado. Show despliega el valor final completo: **`[5 6 4]`**.

Ejercicio 4 — Limitaciones del modelo cliente-servidor en concurrencia declarativa

El problema del merge no determinístico

En el modelo concurrente declarativo, los flujos de datos son **determinísticos por definición**: el valor de una variable solo puede ligarse una vez, y toda comunicación entre procesos ocurre a través de flujos cuyos elementos se producen en orden conocido.

El `contador.oz` implementa un servidor (Counter) que recibe la entrada de múltiples clientes. Para combinar los flujos de los clientes se utiliza la función `Merge`:

```
None
fun {Merge S1 S2}
  case S1
  of nil then S2
  [] X|S1r then X | {Merge S2 S1r}
  end
end
```

¿Qué sucede si un cliente deja de enviar caracteres?

Si Cliente 1 (flujo S1) deja de enviar caracteres pero no cierra su flujo (es decir, S1 queda con la cola sin ligar _), el `case S1` del Merge **se suspende indefinidamente** esperando el próximo elemento de S1. Esto bloquea al servidor aunque Cliente 2 aún tenga datos disponibles. El sistema queda en deadlock parcial.

Para que el servidor continúe, el cliente que deja de enviar debe cerrar explícitamente su flujo con `nil`. En un sistema real esto no siempre es posible.

¿Por qué no es posible el verdadero comportamiento cliente-servidor?

Un servidor real debe poder **atender al cliente que primero esté listo**, sin importar el orden. Esto requiere **merge no determinístico**: elegir entre S1 y S2 dependiendo de cuál tenga datos disponibles primero.

En el modelo concurrente **declarativo** esto es imposible por principio: la elección de qué cliente atender primero dependería de una condición de carrera (qué flujo tiene un elemento

disponible en este momento), lo que introduciría no determinismo observable, violando la semántica declarativa del modelo.

Propiedad	**Modelo declarativo**	**Modelo stateful (celda/objeto)**
Merge	Determinístico (orden fijo)	No determinístico posible
Orden de clientes	Debe conocerse de antemano	Dinámico, en tiempo de ejecución
Bloqueo si cliente falla	Sí	Puede manejarse con timeout
Verdadero cliente-servidor	**Imposible**	Posible

Extensión a N clientes

Se puede extender a N clientes encadenando merges:

```
None
fun {MergeAll Streams}
  case Streams
  of [S] then S
  [] S|Sr then {Merge S {MergeAll Sr}}
  end
end
```

Sin embargo, **la limitación fundamental persiste**: el orden en que los caracteres de los N clientes llegan al servidor queda fijado por la estructura del árbol de merges, no por la disponibilidad real de los datos. Si cualquier cliente intermediario deja de enviar sin cerrar su flujo, todos los clientes "aguas abajo" en el árbol quedan bloqueados.

Ejercicio 5 — Foo y Bar: por qué es necesario el buffer limitado

Descripción del escenario

- **Bar** (Productor): sirve 1 cerveza cada **3 segundos** durante 1 hora → hasta 1200 cervezas.
- **Foo** (Consumidor): bebe 1 cerveza cada **12 segundos** → puede beber hasta 300 cervezas en una hora.
- Restricción del dominio: **máximo 5 cervezas en la mesa al mismo tiempo** (la cerveza se calienta).

¿Por qué el buffer ilimitado no funciona?

Sin restricción, Bar produce cervezas 4 veces más rápido que Foo las consume (12s / 3s = 4). En un flujo sin límite, Bar llevaría una ventaja creciente: después de 1 minuto habría 20 cervezas acumuladas, después de 1 hora unas 900. La cerveza se calentaría, lo que viola la restricción del problema.

En términos del modelo computacional, un buffer ilimitado también es problemático porque el productor nunca se bloquea: ocupa memoria creciente y no hay retroalimentación (*back-pressure*) al productor.

¿Cómo se implementa el buffer limitado en concurrencia declarativa?

Se usa la técnica de **tokens de permiso** (semáforo declarativo):

1. Se generan **N tokens iniciales** (N = 5), representados como un flujo `[token token token token token]`.
2. Bar **consume un token** antes de servir cada cerveza. Si no hay tokens, Bar queda bloqueado (suspendido por dataflow).
3. Foo **devuelve un token** después de beber cada cerveza.
4. Los tokens devueltos se **concatenan** a los iniciales para formar el flujo de permisos total.

```
None
Tokens = [tok tok tok tok tok] ++ DevolucionesDeFoo
Bar: espera token → sirve cerveza → avanza
Foo: bebe cerveza → devuelve token → avanza
```

Cuando hay 5 cervezas en la mesa (Bar consumió los 5 tokens y Foo no ha devuelto ninguno), Bar queda suspendido en `case Tokens of _|...`, esperando que Foo devuelva un token al beber la siguiente cerveza. Esto implementa la restricción de 5 cervezas máximo de forma **puramente declarativa**, sin estado mutable ni primitivas de sincronización imperativas.

Evaluación perezosa vs. evaluación ansiosa

Evaluación ansiosa (por defecto en Oz)

En Oz, las expresiones se evalúan de forma **ansiosa** (*eager*): cada vez que una función es llamada, sus argumentos se evalúan inmediatamente antes de ejecutar el cuerpo de la función. Los hilos creados con `thread ... end` también comienzan a ejecutarse de inmediato (*eager threads*).

```
None
fun {Doble X} X * 2 end
Y = {Doble 3}  % 3 se evalúa inmediatamente, Y = 6
```

Evaluación perezosa (bajo demanda)

Con la anotación `fun lazy`, la función no se ejecuta cuando es invocada. En cambio, se crea un **disparador por necesidad** (*by-need trigger*): una suspensión que se activa solo cuando el valor de la variable es requerido por otro cálculo.

```
None
fun lazy {Gen I} I | {Gen I+1} end
Xs = {Gen 0}   % No se ejecuta nada aún
Y  = Xs.1      % Ahora sí: se necesita la cabeza → Gen ejecuta → Y = 0
```

Diferencias clave

Aspecto	**Ansiosa**	**Perezosa**
Momento de evaluación	Al invocar	Cuando el valor es necesario
Listas infinitas	Imposible (bucle infinito)	Posible (solo se materializa lo necesario)
Consumo de memoria	Toda la estructura	Solo la parte accedida
Concurrencia	Hilo nuevo arranca inmediatamente	Hilo nuevo arranca al necesitarse el valor
Uso típico	Cálculos con resultados acotados	Flujos infinitos, generadores, pipelines

Estructuras de datos potencialmente infinitas

La evaluación perezosa permite definir estructuras de datos **infinitas** que solo se materializan bajo demanda:

```
None
% Lista infinita de enteros desde I
fun lazy {Gen I} I | {Gen I+1} end

% Lista infinita de primos (Criba de Eratóstenes perezosa)
fun lazy {FilterLazy Xs P}
```

```
  case Xs of X|Xr then
    if X mod P \= 0 then X | {FilterLazy Xr P}
    else {FilterLazy Xr P}
    end
  end
end

fun lazy {Sieve Xs}
  case Xs of X|Xr then X | {Sieve {FilterLazy Xr X}} end
end

fun {Primes} {Sieve {Gen 2}} end
```

`{Primes}` representa la lista **infinita** de todos los números primos, pero en memoria solo existe el fragmento que ha sido requerido. Al pedir los primeros N primos con `{ShowPrimes N}`, el sistema materializa exactamente los N primeros elementos y no más.

Ejercicio 9 — Comportamiento concurrente de la evaluación perezosa

Programa analizado

```
None
fun lazy {HagaX} {Browse x} {Delay 3000} 1 end
fun lazy {HagaY} {Browse y} {Delay 6000} 2 end
fun lazy {HagaZ} {Browse z} {Delay 9000} 3 end
X = {HagaX}
Y = {HagaY}
Z = {HagaZ}
{Browse (X+Y)+Z}
```

Explicación del comportamiento

Las funciones `lazy` no se evalúan al ser llamadas; crean **disparadores por necesidad** (*triggers by need*). Solo se ejecutan cuando su valor es requerido.

La evaluación de `{Browse (X+Y)+Z}` procede así:

1. Se necesita `(X+Y)+Z` → se necesita `X+Y` primero.
2. Se necesita `X` → se activa `HagaX` → Browse muestra **x** → espera 3s → X=1.
3. Se necesita `Y` → se activa `HagaY` → Browse muestra **y** → espera 6s → Y=2.

    - x e y aparecen **inmediatamente** (los disparadores se activan en paralelo).
4. Tras 6s: X+Y = 3. Ahora se necesita `Z` → se activa `HagaZ` → Browse muestra **z** → espera 9s → Z=3.
    - z aparece a los **6 segundos**.
5. Tras 6+9 = 15s: Z=3. Resultado = 3+3 = 6. Browse muestra **6** a los **15 segundos**.

Línea de tiempo:

```
None
t=0s   : x e y se muestran (disparadores activados)
t=6s   : z se muestra (Y listo, Z es necesitado)
t=15s  : resultado 6 (Z listo)
```

¿Qué pasa con `X+(Y+Z)`?

```
None
{Browse X+(Y+Z)}
```

1. Se necesita `X+(Y+Z)` → se necesita `X` y `Y+Z`.
2. `X` activa HagaX (muestra x, 3s). `Y+Z` activa HagaY (muestra y, 6s).
3. Pero `Y+Z` necesita Z también → activa HagaZ (muestra z, 9s desde que Y termina, es decir a los 6s).
4. Y listo a 6s, Z listo a 6+9=15s → Y+Z listo a 15s.
5. X listo a 3s, pero X+(Y+Z) necesita Y+Z → espera hasta 15s.
6. Resultado a los **15 segundos**. Igual de lento.

¿Qué pasa con `thread X+Y end + Z`?

```
None
{Browse thread X+Y end + Z}
```

1. `thread X+Y end` crea un hilo **ansioso** que evalúa X+Y en paralelo.
2. Este hilo activa HagaX (muestra x) y HagaY (muestra y) inmediatamente.
3. En el hilo principal se necesita el resultado del thread + Z, lo que activa HagaZ (muestra z).
4. HagaX tarda 3s → X=1. HagaY tarda 6s → Y=2. El hilo termina a los 6s con X+Y=3.
5. HagaZ tarda 9s → Z=3. El hilo principal tiene el resultado del thread (3) y Z (3) → resultado = **6 a los 9 segundos**.

Esta forma es la **más rápida**: 9s vs 15s de las anteriores.

¿Por qué? Porque al poner `X+Y` en un hilo separado (evaluación ansiosa), la suma de X e Y corre en paralelo con la activación y espera de Z. El cuello de botella pasa a ser `max(time(X+Y), time(Z)) = max(6s, 9s) = 9s`.

¿Cómo sumar n enteros $i_1,...,i_n$ disponibles después de $t_1,...,t_n$ ms lo más rápido posible?

La estrategia óptima es usar un **árbol de sumas en paralelo** (divide y vencerás concurrente). En lugar de sumar secuencialmente (que toma `t_max + n·overhead`), se agrupan las sumas en pares dentro de hilos independientes:

```
None
fun {SumaParalela Xs}
  case Xs
  of [X] then X
  [] _ then
    local Izq Der Mid in
      Mid = {List.length Xs} div 2
      thread Izq = {SumaParalela {List.take Xs Mid}} end
      thread Der = {SumaParalela {List.drop Xs Mid}} end
      Izq + Der
    end
  end
end
```

Complejidad: El tiempo total es `max(t ) + O(log n)`, ya que:

- Todas las activaciones de los valores corren en paralelo (se esperan mutuamente).
- Las sumas se hacen en un árbol de profundidad $\log_2(n)$, con cada nivel ejecutándose en paralelo.

Esto es óptimo: no se puede obtener el resultado antes de que el valor más tardío (`max(t )`) esté disponible.